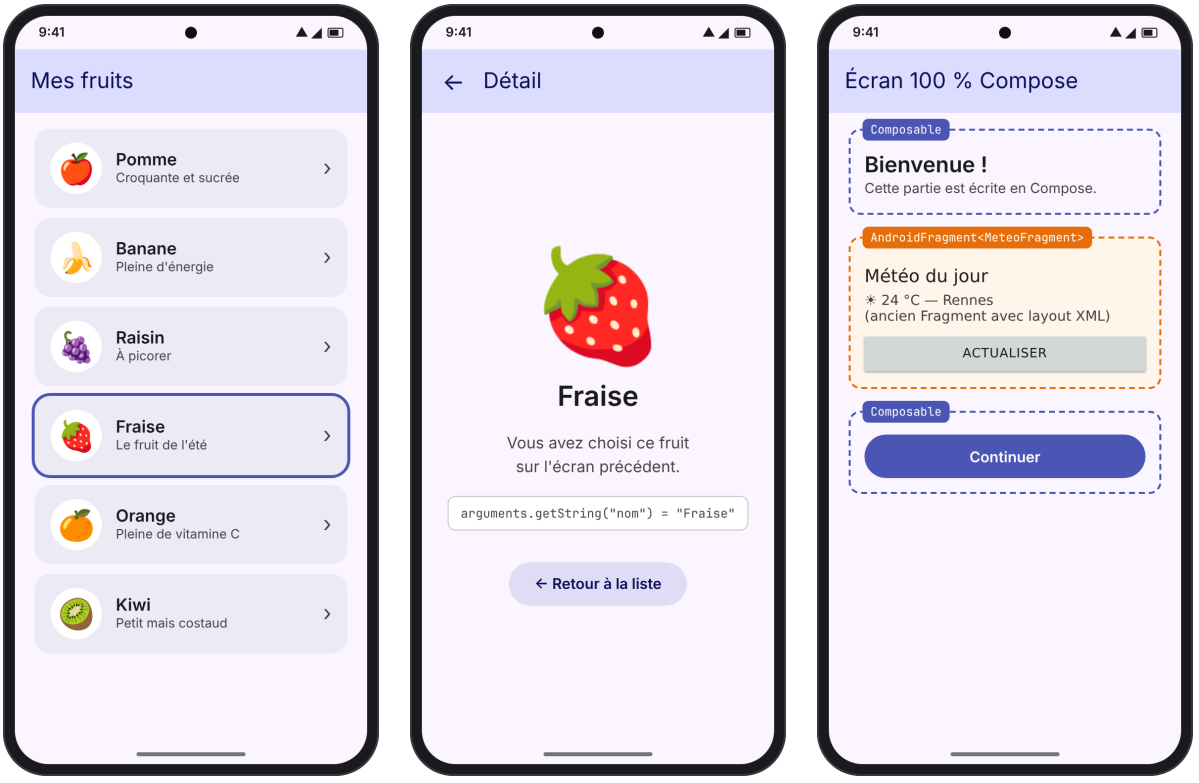
COURS DÉBUTANT · ANDROID

Les Fragments avec Jetpack Compose

Comprendre ce qu'est un Fragment, comment le faire cohabiter avec Compose, et comment s'en passer — avec des exemples simples à copier-coller et des captures d'écran.

Kotlin | Jetpack Compose | Fragments | Navigation



Sommaire

1	Les idées de base : Activity, Fragment, Composable	en 5 minutes
2	Préparer le projet (Gradle)	configuration
3	Exemple 1 — Afficher du Compose dans un Fragment	ComposeView
4	Exemple 2 — Naviguer entre deux Fragments	FragmentManager
5	Exemple 3 — Passer des données (arguments)	Bundle
6	Exemple 4 — Partager des données avec un ViewModel	activityViewModels
7	Exemple 5 — Afficher un ancien Fragment dans Compose	AndroidFragment
8	Exemple 6 — Le 100 % Compose : remplacer les Fragments	Navigation Compose
9	Le cycle de vie, sans se perdre	ViewCompositionStrategy
10	Les erreurs fréquentes du débutant	et leurs solutions
11	Exercices corrigés	pratique
12	Aide-mémoire	à garder sous la main

À qui s'adresse ce cours ?
À toute personne qui connaît les bases de Kotlin (variables, fonctions, classes) et qui a déjà créé un projet dans Android Studio. Aucune connaissance préalable des Fragments n'est nécessaire.

Comment lire ce cours
Chaque exemple est **complet** : vous pouvez copier le code dans un nouveau projet Android Studio (modèle « Empty Activity ») et le lancer. Les captures montrent le résultat attendu à l'écran.

À propos des captures d'écran
Les captures de ce document sont des rendus fidèles des écrans produits par le code de chaque exemple (thème Material 3 par défaut). Selon votre téléphone, votre version d'Android et vos couleurs, l'apparence peut varier légèrement.

CHAPITRE 1

Les idées de base

Avant d'écrire du code, posons trois mots de vocabulaire. Imaginez une application comme une **maison** :

Élément	Dans la maison	Dans Android
Activity	La maison elle-même	Le point d'entrée de l'application, une fenêtre plein écran. En général, une seule suffit.
Fragment	Une pièce de la maison	Un morceau d'écran réutilisable, avec son propre cycle de vie, placé *dans* une Activity. On peut changer de pièce sans quitter la maison.
Composable	Les meubles et la déco	Une fonction Kotlin annotée `@Composable` qui **dessine** l'interface (textes, boutons, listes…).

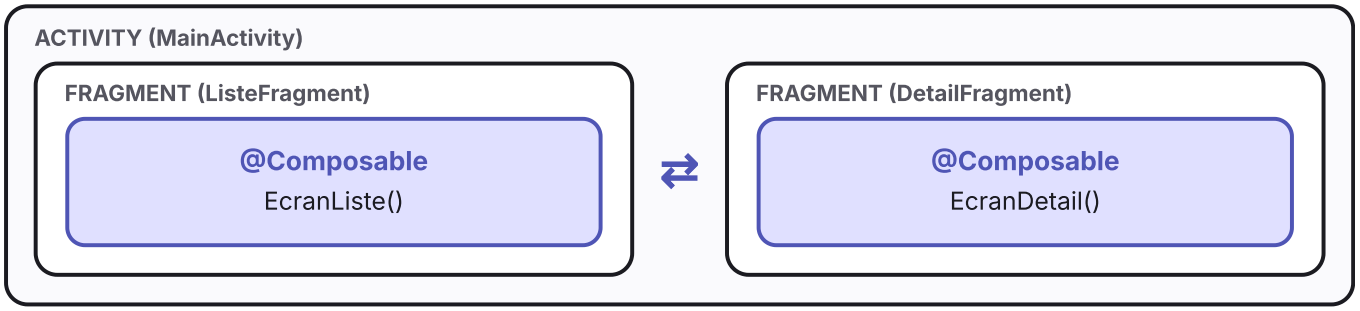


Schéma : une Activity contient des Fragments, et chaque Fragment affiche des Composables.

Fragment ou Compose : faut-il choisir ?

Les Fragments existent depuis 2011, bien avant Compose. Ils ont été pensés pour le système de vues XML. Jetpack Compose, lui, est la façon **moderne** de construire des interfaces Android.

Aujourd'hui, Google recommande pour les **nouvelles applications** d'écrire tous les écrans en Compose, avec **une seule Activity** et **Navigation Compose** — sans Fragment. Mais vous rencontrerez des Fragments partout :

- dans les applications existantes, que l'on migre **petit à petit** vers Compose ;
- dans certaines bibliothèques qui fournissent encore leurs écrans sous forme de Fragment (cartes, paiement, préférences…) ;
- dans les offres d'emploi et les tests techniques !

> **Le bon réflexe**
>
> Savoir **faire cohabiter** Fragments et Compose (chapitres 3 à 7), puis savoir **s'en passer** (chapitre 8). C'est exactement le chemin de ce cours.

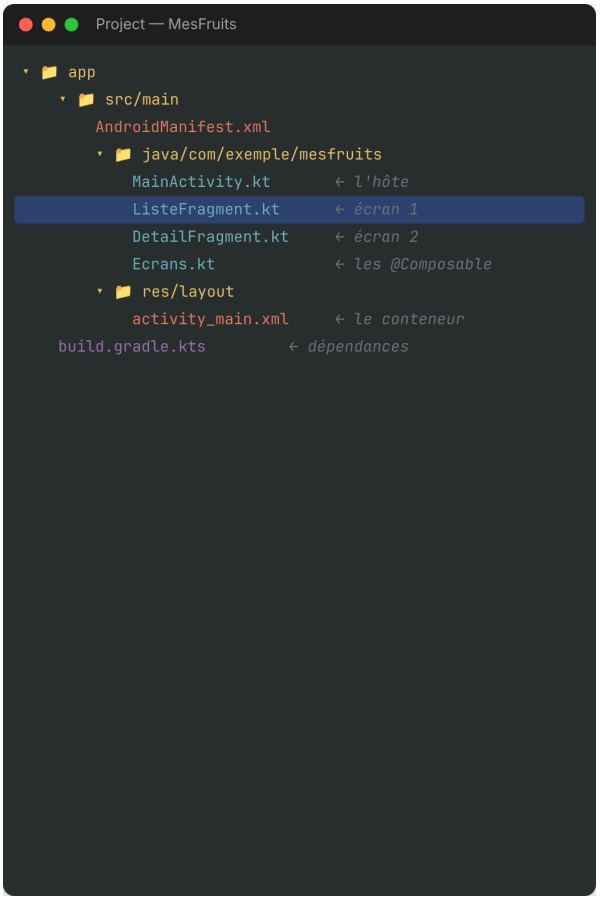
Les trois situations possibles

Situation	Outil à utiliser	Chapitre
J'ai un Fragment et je veux y dessiner avec Compose	`ComposeView` (ou `content { }`)	3 à 6
J'ai un écran Compose et je dois y afficher un ancien Fragment	`AndroidFragment<T>()`	7
Je démarre un projet neuf	Navigation Compose, pas de Fragment	8

CHAPITRE 2

Préparer le projet

Créez un projet dans Android Studio avec le modèle **Empty Activity** (qui active déjà Compose). Nommez-le **MesFruits**. Voici la structure que nous allons obtenir :



Capture 1 — Structure du projet dans Android Studio (vue « Project »).

Ajouter les dépendances

Ouvrez `app/build.gradle.kts` et ajoutez les bibliothèques **Fragment** dans le bloc `dependencies` :

app/build.gradle.kts

```
dependencies {
    // Déjà présentes avec le modèle « Empty Activity »
    implementation(platform("androidx.compose:compose-bom:2024.12.01"))
    implementation("androidx.compose.material3:material3")
    implementation("androidx.activity:activity-compose:1.9.3")

    // À AJOUTER pour les Fragments
    implementation("androidx.fragment:fragment-ktx:1.8.5")       // Fragments + raccourcis
Kotlin
    implementation("androidx.fragment:fragment-compose:1.8.5")   // pont Fragment ⇄ Compose
    implementation("androidx.appcompat:appcompat:1.7.0")         // AppCompatActivity
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
}
```

Les numéros de version

Les versions indiquées fonctionnent ensemble, mais des versions plus récentes existent sûrement quand vous lisez ce cours. Android Studio souligne en jaune les dépendances obsolètes : **Alt + Entrée** propose de les mettre à jour.

Cliquez ensuite sur **Sync Now** (bandeau en haut de l'éditeur) pour télécharger les bibliothèques.

Le thème de l'Activity

Comme nous utiliserons `AppCompatActivity`, l'Activity doit avoir un thème **AppCompat**. Dans `AndroidManifest.xml`, vérifiez la ligne `android:theme` :

app/src/main/AndroidManifest.xml

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:theme="@style/Theme.AppCompat.Light.NoActionBar">
    ...
</activity>
```

Pourquoi « NoActionBar » ?

Parce que la barre de titre sera dessinée par Compose (`TopAppBar`). Sans cela, vous auriez deux barres de titre l'une sur l'autre.

CHAPITRE 3 · EXEMPLE 1

Afficher du Compose dans un Fragment

Objectif : créer un Fragment qui affiche un texte et un compteur de clics écrits en Compose.

Étape 1 — Le conteneur dans l'Activity

Un Fragment a besoin d'un **emplacement** où s'afficher : c'est le `FragmentContainerView`. On le déclare dans un petit fichier XML.

app/src/main/res/layout/activity_main.xml

```
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/conteneur"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Étape 2 — L'Activity hôte

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // On n'ajoute le Fragment qu'au PREMIER lancement
        if (savedInstanceState == null) {
            supportFragmentManager.commit {
                replace(R.id.conteneur, PremierFragment())
            }
        }
    }
}
```

> **Pourquoi tester `savedInstanceState == null` ?**
> Quand on tourne le téléphone, Android recrée l'Activity **et restaure automatiquement** les Fragments. Sans ce test, on en ajouterait un deuxième par-dessus !

Étape 3 — Le Fragment qui affiche du Compose

Un Fragment classique renvoie une `View` dans `onCreateView`. L'astuce : renvoyer une `ComposeView`, une vue spéciale qui sait afficher des Composables.

PremierFragment.kt

```
class PremierFragment : Fragment() {
```

```
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return ComposeView(requireContext()).apply {
            // Libère Compose quand la vue du Fragment est détruite (voir chap. 9)
            setViewCompositionStrategy(
                ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed
            )
            setContent {
                MaterialTheme {
                    EcranPremierFragment()
                }
            }
        }
    }
}
```

Étape 4 — Le Composable

Ecrans.kt

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EcranPremierFragment() {
    // "remember" garde la valeur entre deux dessins
    var clics by remember { mutableIntStateOf(0) }

    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Mon premier Fragment") })
        }
    ) { padding ->
        Column(
            modifier = Modifier
                .padding(padding)
                .fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text("Bonjour ! 👋",
                style = MaterialTheme.typography.headlineMedium)
            Text("Ce texte est dessiné par Jetpack Compose," +
                 " à l'intérieur d'un Fragment.")
            Spacer(Modifier.height(24.dp))
            Text("Nombre de clics")
            Text("$clics",
                style = MaterialTheme.typography.displayLarge)
            Button(onClick = { clics++ }) {
                Text("Cliquer ici")
            }
        }
    }
}
```



Capture 2 — Résultat après 3 clics.

Ce qu'il faut retenir

1. Le **FragmentContainerView** est la « place de parking » du Fragment dans l'Activity.
2. **supportFragmentManager.commit { replace(...) }** place un Fragment dans ce conteneur.
3. Dans le Fragment, **ComposeView + setContent { }** permet d'écrire toute l'interface en Compose.

Raccourci avec fragment-compose

La bibliothèque `fragment-compose` fournit une fonction `content { }` qui crée la `ComposeView` et choisit la bonne stratégie pour vous :

```
import androidx.fragment.compose.content

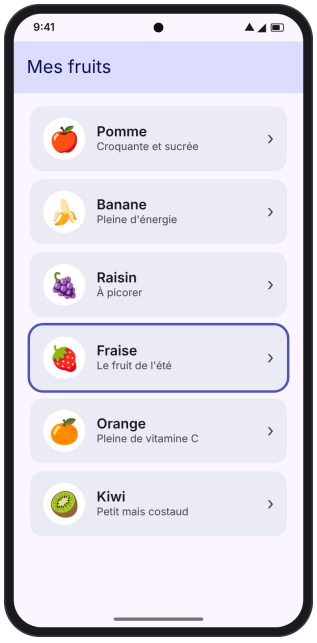
class PremierFragment : Fragment() {
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
                              savedInstanceState: Bundle?) = content {
        MaterialTheme { EcranPremierFragment() }
    }
}
```

Dans la suite du cours, nous utiliserons ce raccourci : c'est plus court et on ne peut pas oublier la stratégie.

CHAPITRE 4 · EXEMPLE 2

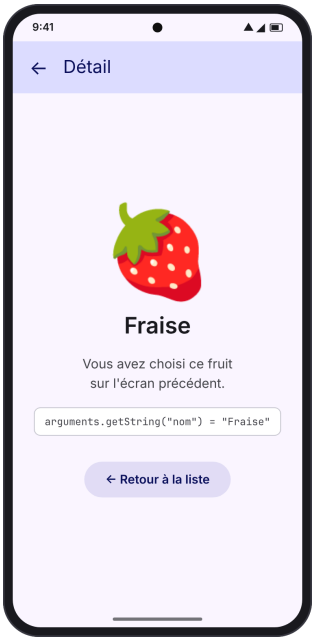
Naviguer entre deux Fragments

Objectif : une liste de fruits (écran 1). Quand on touche un fruit, on ouvre un écran de détail (écran 2). Le bouton « Retour » du téléphone ramène à la liste.



Capture 3 — ListeFragment

clic sur « Fraise »



Capture 4 — DetailFragment

Le principe : la « pile » de retour (back stack)

Le **FragmentManager** range les écrans comme une pile d'assiettes : on pose le nouvel écran **dessus**, et le bouton Retour **retire** l'assiette du dessus.



Le Composable de la liste

Remarquez que le Composable **ne connaît pas** les Fragments : il reçoit simplement une fonction `onFruitClique`. C'est une très bonne habitude, qui permettra de le réutiliser au chapitre 8.

Ecrans.kt

```
data class Fruit(val nom: String, val emoji: String, val description: String)

val fruits = listOf(
    Fruit("Pomme", "🍎", "Croquante et sucrée"),
    Fruit("Banane", "🍌", "Pleine d'énergie"),
    Fruit("Raisin", "🍇", "À picorer"),
    Fruit("Fraise", "🍓", "Le fruit de l'été"),
    Fruit("Orange", "🍊", "Pleine de vitamine C"),
    Fruit("Kiwi", "🥝", "Petit mais costaud"),
```

```
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EcranListe(onFruitClique: (Fruit) -> Unit) {
    Scaffold(topBar = { TopAppBar(title = { Text("Mes fruits") }) }) { padding ->
        LazyColumn(
            contentPadding = padding,
            verticalArrangement = Arrangement.spacedBy(8.dp),
            modifier = Modifier.padding(horizontal = 16.dp)
        ) {
            items(fruits) { fruit ->
                Card(onClick = { onFruitClique(fruit) }, modifier = Modifier.fillMaxWidth()) {
                    Row(Modifier.padding(16.dp), verticalAlignment =
Alignment.CenterVertically) {
                        Text(fruit.emoji, fontSize = 32.sp)
                        Spacer(Modifier.width(16.dp))
                        Column {
                            Text(fruit.nom, style = MaterialTheme.typography.titleMedium)
                            Text(fruit.description, style =
MaterialTheme.typography.bodySmall)
                        }
                    }
                }
            }
        }
    }
}
```

Le Fragment de la liste

C'est ici que l'on fait le lien : quand un fruit est cliqué, on demande au FragmentManager d'afficher le `DetailFragment`.

ListeFragment.kt

```
class ListeFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
                              savedInstanceState: Bundle?) = content {
        MaterialTheme {
            EcranListe(onFruitClique = { fruit -> ouvrirDetail(fruit.nom) })
        }
    }

    private fun ouvrirDetail(nom: String) {
        parentFragmentManager.commit {
            setReorderingAllowed(true)                          // bonne pratique
            replace(R.id.conteneur, DetailFragment.nouveau(nom))   // on remplace l'écran
            addToBackStack(null)                                // pour que « Retour » marche
        }
    }
}
```

Oubli classique : addToBackStack

Sans `addToBackStack(null)`, l'appui sur Retour **ferme l'application** au lieu de revenir à la liste, car la transaction n'a pas été mémorisée.

Et dans `MainActivity`, on affiche maintenant `ListeFragment()` au démarrage :

```
MainActivity.kt (extrait)
```

```
if (savedInstanceState == null) {
    supportFragmentManager.commit { replace(R.id.conteneur, ListeFragment()) }
}
```

Le Fragment de détail

Le `DetailFragment` est présenté en détail au chapitre suivant, car il doit **recevoir** le nom du fruit. Pour le bouton « Retour » dessiné à l'écran, une seule ligne suffit :

```
// Simule l'appui sur le bouton Retour du téléphone
requireActivity().onBackPressedDispatcher.onBackPressed()

// ou, directement avec le FragmentManager :
parentFragmentManager.popBackStack()
```

Méthode	Effet
`add(id, fragment)`	Ajoute le Fragment **par-dessus** l'actuel (les deux existent).
`replace(id, fragment)`	Retire l'actuel et met le nouveau à sa place. **Le plus utilisé.**
`remove(fragment)`	Retire un Fragment.
`addToBackStack(nom)`	Mémorise la transaction pour que « Retour » l'annule.
`popBackStack()`	Annule la dernière transaction mémorisée (= Retour).

CHAPITRE 5 · EXEMPLE 3

Passer des données à un Fragment

Objectif : envoyer le nom du fruit choisi au `DetailFragment`.

> **À ne JAMAIS faire**
> Passer les données par le constructeur : `DetailFragment("Fraise")`. Quand Android recrée le Fragment (rotation, retour d'arrière-plan…), il appelle le constructeur **vide** : votre donnée est perdue et l'application plante.

La bonne méthode : ranger les données dans les **arguments** du Fragment, un « sac » (`Bundle`) qu'Android sauvegarde et restaure automatiquement.

DetailFragment.kt

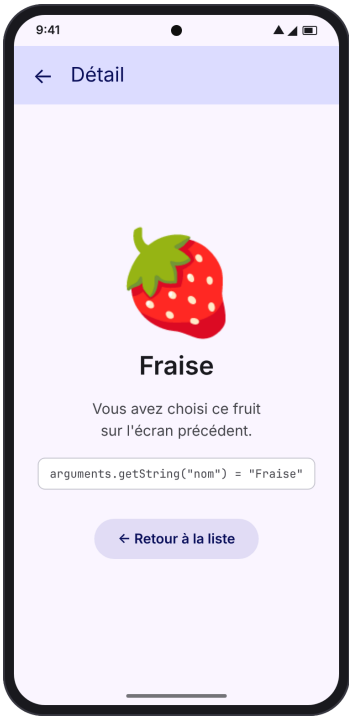
```
class DetailFragment : Fragment() {

    companion object {
        private const val CLE_NOM = "nom"

        // "Usine" : la seule façon de créer
        // le Fragment avec ses données
        fun nouveau(nom: String) = DetailFragment().apply {
            arguments = bundleOf(CLE_NOM to nom)
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ) = content {
        // On relit la donnée dans les arguments
        val nom = requireArguments().getString(CLE_NOM)!!
        val fruit = fruits.first { it.nom == nom }

        MaterialTheme {
            EcranDetail(
                fruit = fruit,
                onRetour = { parentFragmentManager.popBackStack()
}
            )
        }
    }
}
```



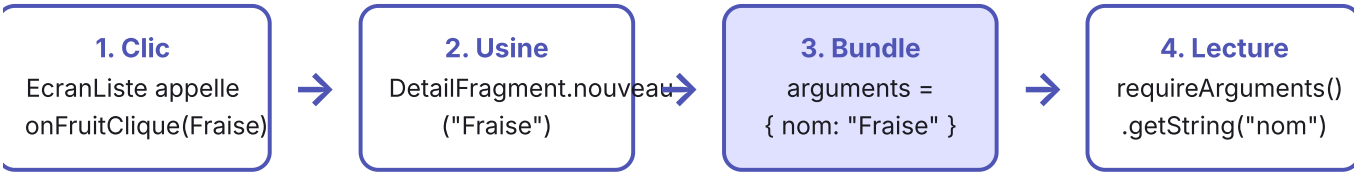
Capture 5 — Le nom « Fraise » est bien reçu par le Fragment.

Ecrans.kt

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EcranDetail(fruit: Fruit, onRetour: () -> Unit) {
    Scaffold(
```

```
        topBar = {
            TopAppBar(
                title = { Text("Détail") },
                navigationIcon = {
                    IconButton(onClick = onRetour) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Retour")
                    }
                }
            )
        }
    ) { padding ->
        Column(
            Modifier.padding(padding).fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text(fruit.emoji, fontSize = 96.sp)
            Text(fruit.nom, style = MaterialTheme.typography.headlineMedium)
            Text("Vous avez choisi ce fruit sur l'écran précédent.")
            Spacer(Modifier.height(16.dp))
            FilledTonalButton(onClick = onRetour) { Text("← Retour à la liste") }
        }
    }
}
```

Ce qui se passe, étape par étape



Quels types peut-on mettre dans un Bundle ?

Les types simples : `String`, `Int`, `Boolean`, `Long`, `Double`, leurs listes et tableaux… Pour un objet complet, passez plutôt son **identifiant** (ex. `id = 42`) et retrouvez l'objet dans votre base de données ou votre ViewModel. Le Bundle doit rester **petit**.

Et pour renvoyer un résultat à l'écran précédent ?

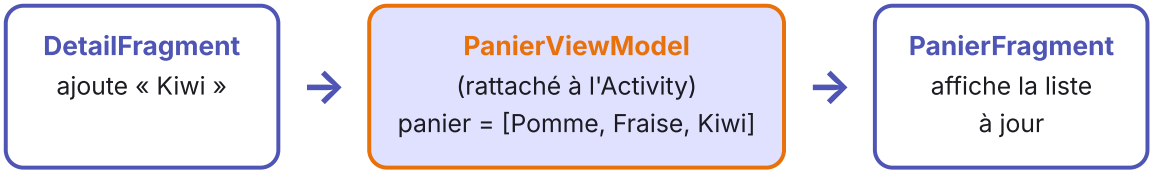
On utilise l'API *Fragment Result* : `setFragmentResult("choix", bundleOf("nom" to "Kiwi"))` dans le Fragment qui répond, et `setFragmentResultListener("choix") { _, bundle → ... }` dans celui qui attend. Pour un débutant, le ViewModel partagé (chapitre suivant) est souvent plus simple.

CHAPITRE 6 · EXEMPLE 4

Partager des données avec un ViewModel

Objectif : un panier de fruits. On ajoute un fruit depuis l'écran de détail, et le panier se met à jour automatiquement sur un autre Fragment.

Un **ViewModel** est une classe qui garde les données en mémoire, même quand l'écran tourne. Si deux Fragments demandent le ViewModel **de l'Activity**, ils reçoivent **le même objet** : c'est une boîte partagée.



PanierViewModel.kt

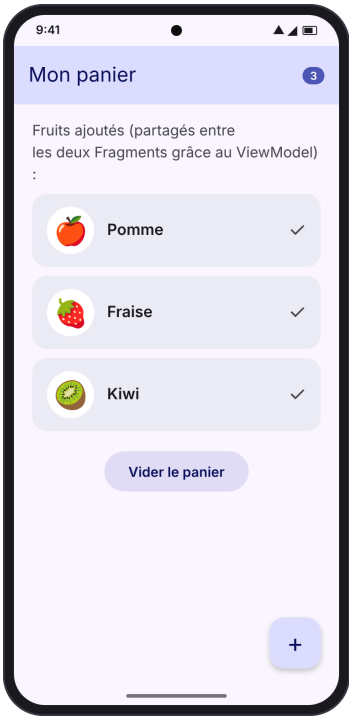
```kotlin
class PanierViewModel : ViewModel() {
    // Liste observable par Compose
    val panier = mutableStateListOf<Fruit>()

    fun ajouter(fruit: Fruit) {
        if (fruit !in panier) panier.add(fruit)
    }
    fun vider() = panier.clear()
}
```

PanierFragment.kt

```kotlin
class PanierFragment : Fragment() {
    // "activityViewModels" = partagé
    // avec tous les Fragments de l'Activity
    private val vm: PanierViewModel
        by activityViewModels()

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ) = content {
        MaterialTheme {
            EcranPanier(
                fruits = vm.panier,
                onVider = vm::vider
            )
        }
    }
}
```



Capture 6 — Le panier, partagé entre deux Fragments.

Dans `DetailFragment`, il suffit de déclarer le même ViewModel et d'appeler `vm.ajouter(fruit)` dans un bouton :

DetailFragment.kt (extrait)

```kotlin
private val vm: PanierViewModel by activityViewModels()
// ...
Button(onClick = { vm.ajouter(fruit) }) { Text("Ajouter au panier") }
```

Déclaration	Durée de vie / partage
`by viewModels()`	Propre à **ce** Fragment uniquement.
`by activityViewModels()`	Partagé par **tous** les Fragments de l'Activity.

CHAPITRE 7 · EXEMPLE 5

Afficher un ancien Fragment dans Compose

Situation : votre écran est écrit en Compose, mais vous devez y intégrer un Fragment existant (par exemple un vieux `MeteoFragment` en XML, ou un Fragment fourni par une bibliothèque). C'est le chemin **inverse** des chapitres précédents.

La bibliothèque `fragment-compose` fournit pour cela le Composable `AndroidFragment`.

MainActivity.kt

```
// Ici, plus besoin de layout XML :
// l'Activity affiche directement du Compose.
class MainActivity : AppCompatActivity() {
    override fun onCreate(s: Bundle?) {
        super.onCreate(s)
        setContent {
            MaterialTheme { EcranAccueil() }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun EcranAccueil() {
    Scaffold(topBar = {
        TopAppBar(title = { Text("Écran 100 % Compose") })
    }) { padding ->
        Column(Modifier.padding(padding).padding(16.dp)) {
            Text("Bienvenue !",
                style =
MaterialTheme.typography.headlineSmall)

            // ⬇ Le Fragment, comme un Composable
            AndroidFragment<MeteoFragment>(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(180.dp),
                arguments = bundleOf("ville" to "Rennes")
            )

            Button(onClick = { /* ... */ },
                   modifier = Modifier.fillMaxWidth()) {
                Text("Continuer")
            }
        }
    }
}
```



Capture 7 — En orange, le Fragment intégré au milieu d'un écran Compose.

Bon à savoir

- `AndroidFragment` crée et gère le Fragment **pour vous** : pas de `FragmentManager`, pas de `commit`.
- Les `arguments` sont transmis au Fragment exactement comme au chapitre 5 : il les lit avec `requireArguments()`.
- Le paramètre `onUpdate = { fragment -> ... }` permet d'appeler des méthodes du Fragment une fois qu'il est créé.
- L'Activity doit hériter de `FragmentActivity` (ou `AppCompatActivity`, qui en hérite).

Ancienne méthode

Sur Internet, vous trouverez encore `AndroidViewBinding` avec un `FragmentContainerView` dans un XML. Cela fonctionne, mais `AndroidFragment` est aujourd'hui la solution recommandée, beaucoup plus simple.

CHAPITRE 8 · EXEMPLE 6

Le 100 % Compose : remplacer les Fragments

Pour une **nouvelle application**, on n'utilise plus de Fragments du tout. Chaque écran est un simple Composable, et c'est **Navigation Compose** qui gère la pile d'écrans et le bouton Retour.

Avec Fragments	Avec Navigation Compose
Une classe `Fragment` par écran	Une fonction `@Composable` par écran
`FragmentContainerView`	`NavHost`
`FragmentManager.commit { replace(...) }`	`navController.navigate(...)`
`addToBackStack()`	Automatique
Arguments dans un `Bundle`	Une `data class` typée
`popBackStack()`	`navController.popBackStack()`

Configuration

app/build.gradle.kts

```
plugins {
    // ... plugins existants
    kotlin("plugin.serialization") version "2.0.21"   // même version que votre Kotlin
}
dependencies {
    implementation("androidx.navigation:navigation-compose:2.8.5")
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
}
```

Étape 1 — Déclarer les destinations (« routes »)

Routes.kt

```
import kotlinx.serialization.Serializable

@Serializable object Accueil                          // écran sans paramètre
@Serializable data class Detail(val nom: String)     // écran avec un paramètre
```

> **Le gros avantage**
> Fini les clés `"nom"` tapées à la main dans un Bundle : si vous vous trompez de type ou oubliez un paramètre, le **compilateur** vous prévient avant même de lancer l'application.

Étape 2 — Le NavHost

MainActivity.kt

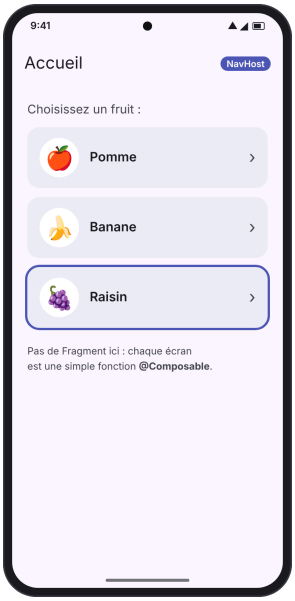
```
class MainActivity : ComponentActivity() {      // plus besoin d'AppCompatActivity
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MaterialTheme { MonApplication() } }
    }
}

@Composable
fun MonApplication() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Accueil) {

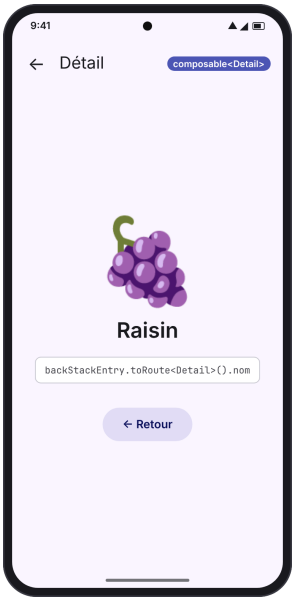
        composable<Accueil> {
            // On RÉUTILISE le Composable du chapitre 4 !
            EcranListe(onFruitClique = { fruit ->
                navController.navigate(Detail(nom = fruit.nom))
            })
        }

        composable<Detail> { backStackEntry ->
            val route: Detail = backStackEntry.toRoute()
            val fruit = fruits.first { it.nom == route.nom }
            EcranDetail(fruit = fruit, onRetour = { navController.popBackStack() })
        }
    }
}
```



Capture 8 — Destination `Accueil`

navigate(
Detail("Raisin"))



Capture 9 — Destination `Detail`

Vous voyez la différence ?

Les deux Composables `EcranListe` et `EcranDetail` n'ont **pas changé d'une ligne**. Nous avons seulement supprimé les classes `ListeFragment`, `DetailFragment`, le fichier XML et le FragmentManager. C'est pour cela qu'on conseille d'écrire des Composables qui reçoivent des **fonctions** (`onFruitClique`, `onRetour`) plutôt que de manipuler la navigation eux-mêmes : la migration devient un jeu d'enfant.

Migrer une application existante, pas à pas

1. Dans chaque Fragment, remplacer le layout XML par `content { }` + un Composable (chapitre 3).
2. Sortir la logique de navigation des Composables : ils reçoivent des lambdas.
3. Quand **tous** les écrans sont des Composables, créer un `NavHost` et une destination par écran.
4. Supprimer les classes Fragment, devenues inutiles. Garder `AndroidFragment` seulement pour les Fragments de bibliothèques.

CHAPITRE 9

Le cycle de vie, sans se perdre

Un Fragment passe par plusieurs étapes, un peu comme une pièce que l'on construit, utilise, puis démonte. Voici les principales :



Méthode	Quand ?	Que faire dedans ?
`onCreate`	Le Fragment est créé	Lire les arguments, préparer des données (pas d'interface).
`onCreateView`	Il faut dessiner l'écran	**Renvoyer la ComposeView** (c'est ce que nous faisons depuis le début).
`onResume`	L'écran est visible et actif	Rarement utile avec Compose.
`onDestroyView`	L'écran est retiré (ex. on passe au Fragment suivant)	Compose est libéré automatiquement grâce à la stratégie ci-dessous.

> **Piège important : la vue meurt avant le Fragment**
> Quand on navigue de la liste vers le détail avec `addToBackStack`, le `ListeFragment` **n'est pas détruit** (il attend dans la pile) mais sa **vue** l'est. C'est pourquoi on utilise la stratégie `DisposeOnViewTreeLifecycleDestroyed` : Compose est nettoyé en même temps que la vue, ni trop tôt, ni trop tard. La fonction `content { }` l'applique automatiquement.

Les stratégies de ViewCompositionStrategy

Stratégie	Quand l'utiliser
`DisposeOnViewTreeLifecycleDestroyed`	**Dans un Fragment.** Le bon choix 9 fois sur 10.
`DisposeOnDetachedFromWindowOrReleasedFromPool`	Valeur par défaut. Adaptée à une ComposeView dans une Activity ou une liste RecyclerView.
`DisposeOnLifecycleDestroyed(lifecycle)`	Cas avancés, lorsqu'on veut choisir précisément le cycle de vie.

Et l'état de l'écran ?

- `remember { }` garde une valeur tant que le Composable est affiché, mais **la perd** si l'on tourne l'écran ou si la vue du Fragment est détruite.
- `rememberSaveable { }` survit à la rotation et au retour depuis la pile.

- Un **ViewModel** garde des données plus importantes (listes, résultats réseau…) et survit à la rotation.

```
// Le compteur du chapitre 3 survit maintenant à la rotation :
var clics by rememberSaveable { mutableIntStateOf(0) }
```

CHAPITRE 10

Les erreurs fréquentes du débutant

1. « IllegalArgumentException: No view found for id ... »

Le FragmentManager ne trouve pas le conteneur.
Solution : vérifiez que `setContentView(R.layout.activity_main)` est appelé **avant** `commit`, et que l'id (`R.id.conteneur`) est le bon.

2. Deux Fragments superposés après une rotation

Solution : entourez l'ajout initial par `if (savedInstanceState == null) { … }` (chapitre 3).

3. « Fragment ... could not be instantiated / no empty constructor »

Vous avez ajouté des paramètres au constructeur du Fragment.
Solution : constructeur vide + `arguments` + fonction usine `nouveau(...)` (chapitre 5).

4. Le bouton Retour ferme l'application

Solution : ajoutez `addToBackStack(null)` dans la transaction (chapitre 4).

5. « You need to use a Theme.AppCompat theme »

Votre Activity hérite de `AppCompatActivity` mais son thème n'est pas AppCompat.
Solution : utilisez `@style/Theme.AppCompat.Light.NoActionBar` dans le manifeste (chapitre 2).

6. « Unresolved reference: commit / content / AndroidFragment »

Il manque une dépendance ou un import.
Solution : vérifiez `fragment-ktx` et `fragment-compose` dans Gradle, puis importez `androidx.fragment.app.commit` et `androidx.fragment.compose.content` / `AndroidFragment`.
Dans Android Studio, **Alt + Entrée** sur le mot en rouge ajoute l'import.

7. Les données disparaissent quand je reviens en arrière

Vous utilisez `remember` : l'état est perdu quand la vue du Fragment est détruite.
Solution : `rememberSaveable` ou un ViewModel (chapitre 9).

8. Le Composable navigue lui-même vers un Fragment

Appeler `parentFragmentManager` depuis un Composable rend le code difficile à réutiliser.
Solution : le Composable reçoit un `onClic: () → Unit`, et c'est le Fragment qui décide quoi faire (chapitres 4 et 8).

CHAPITRE 11

Exercices corrigés

EXERCICE 1 · FACILE

Créez un Fragment `BonjourFragment` qui affiche, en Compose, le texte « Bonjour, <prénom> ! ». Le prénom doit être passé en argument.

Correction

```
class BonjourFragment : Fragment() {
    companion object {
        fun nouveau(prenom: String) = BonjourFragment().apply {
            arguments = bundleOf("prenom" to prenom)
        }
    }
    override fun onCreateView(i: LayoutInflater, c: ViewGroup?, s: Bundle?) = content {
        val prenom = requireArguments().getString("prenom")
        MaterialTheme { Text("Bonjour, $prenom !", style =
MaterialTheme.typography.headlineLarge) }
    }
}
// Dans MainActivity :
supportFragmentManager.commit { replace(R.id.conteneur, BonjourFragment.nouveau("Léa")) }
```

EXERCICE 2 · MOYEN

Dans l'application des fruits, ajoutez sur l'écran de détail un bouton « Voir un autre fruit au hasard » qui ouvre un nouveau `DetailFragment` avec un fruit aléatoire. Que se passe-t-il quand on appuie plusieurs fois sur Retour ?

Correction

```
// Dans DetailFragment, on passe une nouvelle lambda à EcranDetail :
onHasard = {
    val autre = fruits.random()
    parentFragmentManager.commit {
        setReorderingAllowed(true)
        replace(R.id.conteneur, DetailFragment.nouveau(autre.nom))
        addToBackStack(null)
    }
}
```

Réponse : chaque clic empile un nouveau détail. Le bouton Retour revient donc sur **chaque** fruit vu précédemment, dans l'ordre inverse, avant de revenir à la liste.

EXERCICE 3 · DÉFI

Convertissez l'exercice 2 en **Navigation Compose** (chapitre 8), sans aucun Fragment.

Correction

```
composable<Detail> { entry ->
    val fruit = fruits.first { it.nom == entry.toRoute<Detail>().nom }
    EcranDetail(
        fruit = fruit,
        onRetour = { navController.popBackStack() },
        onHasard = { navController.navigate(Detail(fruits.random().nom)) }
    )
}
```

Le comportement de la pile est identique : Navigation Compose empile les destinations comme le faisait `addToBackStack`.

CHAPITRE 12

Aide-mémoire

Compose dans un Fragment

```
override fun onCreateView(i: LayoutInflater, c: ViewGroup?, s: Bundle?) = content {
    MaterialTheme { MonEcran() }
}
```

Afficher / remplacer un Fragment

```
parentFragmentManager.commit {
    setReorderingAllowed(true)
    replace(R.id.conteneur, AutreFragment.nouveau("valeur"))
    addToBackStack(null)
}
```

Arguments

```
MonFragment().apply { arguments = bundleOf("cle" to "valeur") }   // envoyer
requireArguments().getString("cle")                               // lire
```

ViewModel partagé

```
private val vm: MonViewModel by activityViewModels()
```

Fragment dans Compose

```
AndroidFragment<MonFragment>(modifier = Modifier.fillMaxSize(), arguments = bundleOf("cle" to
1))
```

100 % Compose

```
NavHost(navController, startDestination = Accueil) {
    composable<Accueil> { EcranAccueil(onClic = { navController.navigate(Detail("x")) }) }
    composable<Detail>  { EcranDetail(it.toRoute<Detail>().nom) }
}
```

Check-list avant de lancer l'application

- ☐ Les dépendances `fragment-ktx` et `fragment-compose` sont ajoutées et synchronisées.
- ☐ L'Activity hérite de `AppCompatActivity` (ou `FragmentActivity`) et a un thème AppCompat.
- ☐ Le Fragment est ajouté seulement si `savedInstanceState == null`.
- ☐ Les Fragments ont un constructeur vide ; les données passent par les `arguments`.
- ☐ Chaque navigation qui doit être annulable a son `addToBackStack`.

☐ Les Composables reçoivent des lambdas et ne connaissent pas les Fragments.

Pour aller plus loin

Documentation officielle Android (developer.android.com) : pages « Fragments », « Interopérabilité Compose et Views », « Navigation avec Compose » et le codelab « Migrer vers Jetpack Compose ».

Bravo, vous savez maintenant faire cohabiter Fragments et Compose — et vous en passer ! 🎉